

Event Groups (Bitwise Synchronization)

Up until now, we have used Semaphores and Queues to unlock a task based on a single event (e.g., data arrives in a queue, or a mutex key becomes available).

But real-world embedded systems often require a task to wait for multiple conditions to be true simultaneously before it is allowed to run. For example, think of a smart device dashboard: it shouldn't initialize its screen until:

1. The Wi-Fi radio connects successfully AND
2. The System Time is synchronized from the internet AND
3. The internal SD Card memory is mounted.

If we used semaphores for this, your code would become a messy, nested pile of `xSemaphoreTake()` statements. Event Groups solve this cleanly by letting a task sleep until a specific pattern of binary bits (flags) are set.

How Event Groups Work

An Event Group is essentially a collection of binary flags (bits) stored within a single variable inside the RTOS kernel.

- **The Event Bits:** You define what each bit represents using basic C preprocessor macros (e.g., `BIT_0` for Wi-Fi, `BIT_1` for Time, `BIT_2` for SD Card).
- **Setting Bits:** Any task (or interrupt) can set a bit when an action completes using `xEventGroupSetBits()`.
- **Waiting for Bits:** A controller task can call `xEventGroupWaitBits()` to go to sleep until either **ALL** specified bits are set (AND logic), or **ANY** one of the bits is set (OR logic).

Code:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"

#define WIFI_CONNECTED_BIT (1<<0) // Bit 0 (0x01)
#define SENSORS_READY_BIT (1<<1) // Bit 1 (0x02)

// Global event group handle
EventGroupHandle_t system_event_group;
// Simulating connecting to a Wi-Fi network asynchronously
void wifi_setup_task(void *pvParameters){
    printf("[Wifi Setup] Starting Wi-Fi driver module... \n");
```

```

    vTaskDelay(2000 / portTICK_PERIOD_MS); // Simulating 2 seconds of
connection time

    printf("[WiFi Setup] Connected successfully! Setting event bit. \n");

    // Setting the Wi-Fi bit in our event group
    xEventGroupSetBits(system_event_group, WIFI_CONNECTED_BIT);

    // Deleting this task since its setup job is done
    vTaskDelete(NULL);
}
// Simulating calibrating hardware sensors
void sensor_setup_task(void *pvParameters){
    printf("[Sensor Setup] Calibrating peripheral hardware arrays... \n");
    vTaskDelay(4000 / portTICK_PERIOD_MS);

    printf("[Sensor Setup] Calibration complete! Setting event bit. \n");
    // Setting the sensors bit in our event group
    xEventGroupSetBits( system_event_group, SENSORS_READY_BIT);

    vTaskDelete(NULL);
}
// The master control system waits for authorization
void master_controller_task(void *pvParameters){
    printf("[Master Core] Standing by. Waiting for all subsystems to
clear...\n");

    // Block here until BOTH bits are set
    EventBits_t bits = xEventGroupWaitBits(
        system_event_group,
        WIFI_CONNECTED_BIT | SENSORS_READY_BIT,
        pdTRUE,
        pdTRUE,
        portMAX_DELAY
    );

    printf("[Master Core] SUCCESS! Received bitmask: 0x%02X. System booting
initialized! \n", (unsigned int)bits);

    while(1){
        printf("[Master Core] Operational Loop Running smoothly...\n");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

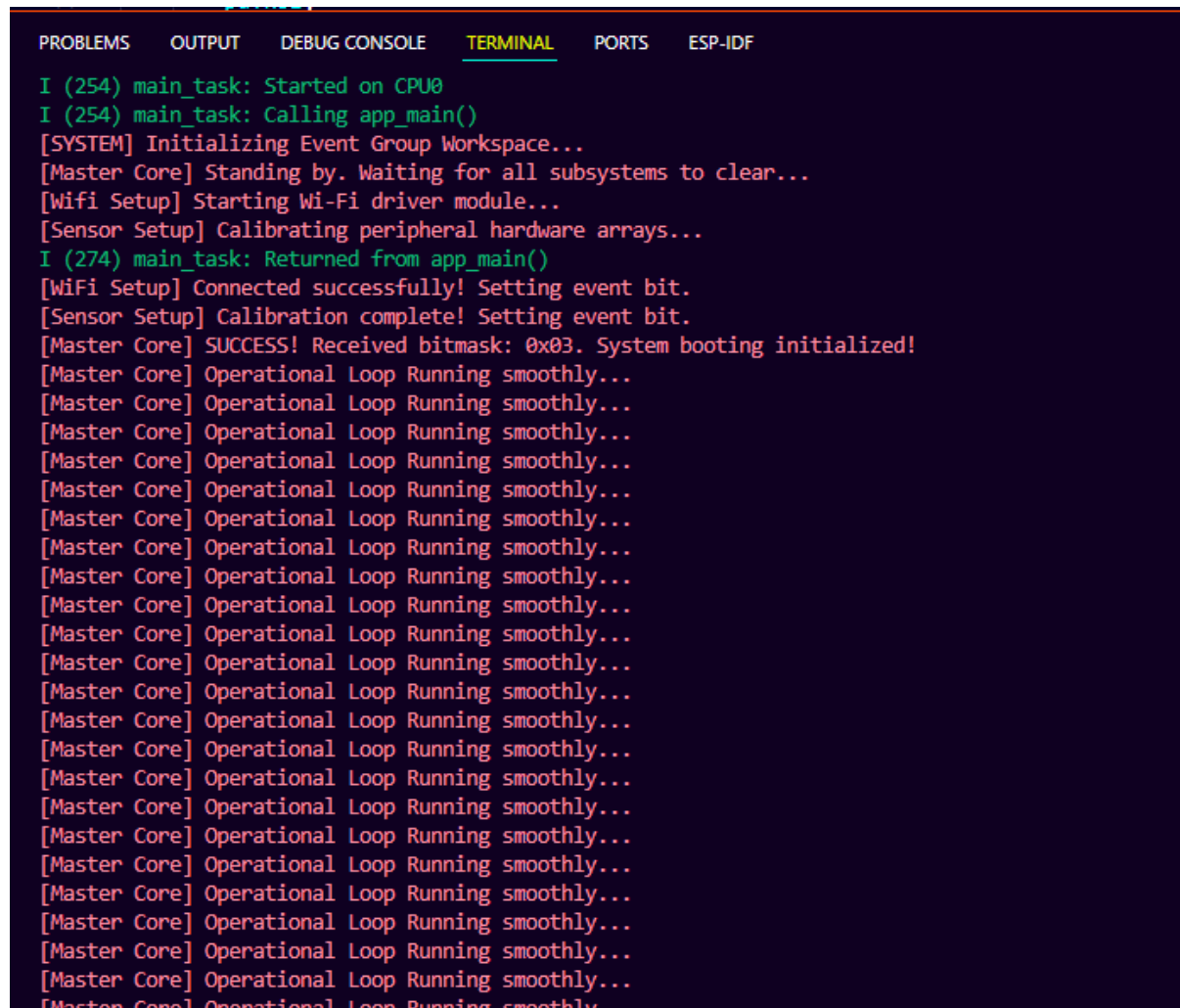
void app_main(void){
    printf("[SYSTEM] Initializing Event Group Workspace...\n");

```

```
// Creating the event group structure
system_event_group = xEventGroupCreate();

if(system_event_group != NULL){
    xTaskCreate(master_controller_task, "Master", 2048, NULL, 5, NULL);
    xTaskCreate(wifi_setup_task, "WiFi Boot", 2048, NULL, 5, NULL);
    xTaskCreate(sensor_setup_task, "Sensor Boot", 2048, NULL, 5, NULL);
}
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF

I (254) main_task: Started on CPU0
I (254) main_task: Calling app_main()
[SYSTEM] Initializing Event Group Workspace...
[Master Core] Standing by. Waiting for all subsystems to clear...
[Wifi Setup] Starting Wi-Fi driver module...
[Sensor Setup] Calibrating peripheral hardware arrays...
I (274) main_task: Returned from app_main()
[WiFi Setup] Connected successfully! Setting event bit.
[Sensor Setup] Calibration complete! Setting event bit.
[Master Core] SUCCESS! Received bitmask: 0x03. System booting initialized!
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
[Master Core] Operational Loop Running smoothly...
```

- ☐ The Master Core immediately boots and freezes in a blocked state.
- ☐ At 2 seconds, the Wi-Fi task sets its bit and dies. The Master remains blocked.
- ☐ At 4 seconds, the Sensor task sets its bit and dies.
- ☐ The exact instant the final bit lands, the Master Core wakes up instantly and starts printing its execution loop.

Seeing 0x03 proves that the master task held its ground in a low-power blocked state until exactly **Bit 0** (WIFI_CONNECTED_BIT = 1) and **Bit 1** (SENSORS_READY_BIT = 2) were both slammed high by the setup workers. $1 + 2 = 3$ (or 0x03 in hex).